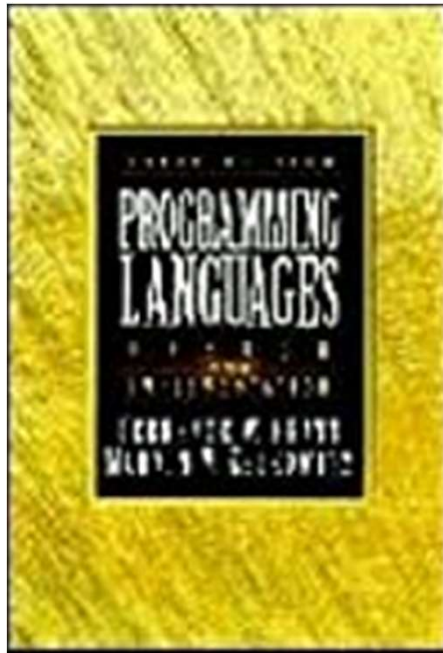


Controle de Sequência

Capítulo 6, Pratt & Zelkowitz

1



- Controle de sequência implícito e explícito.
- Sequenciamento com expressões aritméticas.
- Controle de sequência em comandos

Estruturas de controle

2

- provêm arcabouço para combinar operações e dados em programas e conjunto de programas.
- aspectos da combinação de dados e operações:
 - controle da sequência de execução das operações (Capítulo 6)
 - controle da transmissão de dados entre subprogramas de um programa (controle de dados) (Capítulo 7)

Controle de Sequência Implícito e Explícito

3

- Grupos de estruturas de controle de sequência
 - ▣ usadas em **expressões** (dentro de comandos), p. ex.:
 - regras de precedência, associatividade e parênteses.
 - ▣ estruturas usadas **entre (grupos de) comandos**, p. ex.:
 - comandos de iteração e condicionais
 - ▣ estruturas usadas **entre subprogramas**, p. ex.:
 - chamada de subprogramas,
 - corotinas,
 - outras formas de comunicação entre subprogramas.

Controle de sequência Implícito e Explícito

4

- Controle **implícito**: estruturas definidas pela LP
 - ▣ ordem física da sequência dos comandos,
 - ▣ precedência das operações em expressões.
- Controle **explícito**:
 - ▣ estruturas de controle inseridas pelo programador para modificar a sequência implícita de operações
 - comando go to e rótulos de comandos,
 - uso de parênteses em expressões,
 - comandos de iteração e condicionais.

Sequenciamento com Expressões Aritméticas

5

- Qual a ordem de execução das operações?

$$r = \frac{-b \pm \sqrt{b^2 - 4 \times a \times c}}{2 \times a}$$

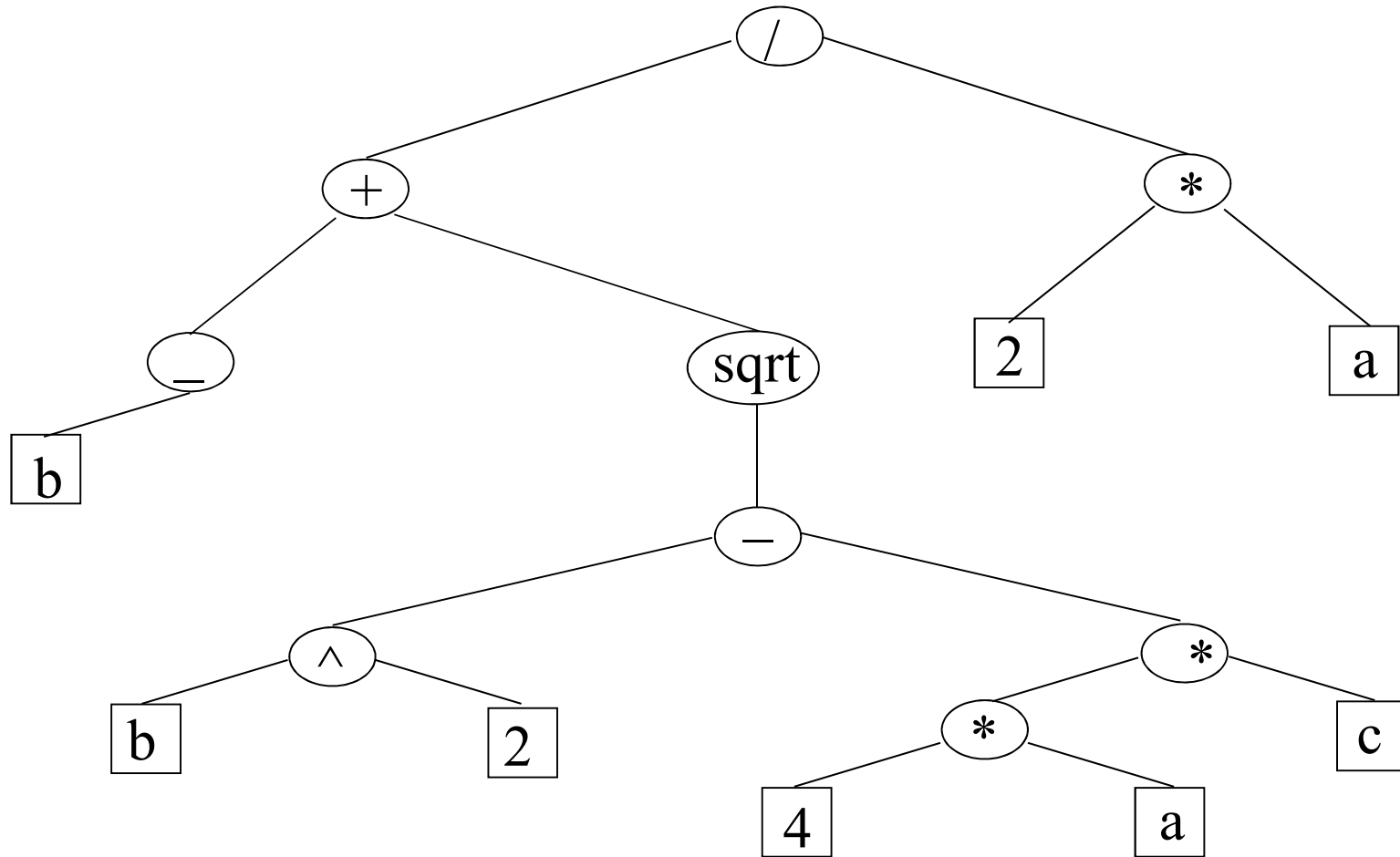
- O controle básico é a **composição funcional**:
 - ▣ as operações e seus operandos são especificados,
 - ▣ um operando pode ser uma constante, OD ou outra operação, e assim por diante, em qualquer nível,
 - ▣ uma estrutura de árvore para a expressão aritmética é induzida pela composição funcional, com a definição da prioridade e associatividade das operações.

Em Assembly a codificação da fórmula requer 15 ou mais instruções.

Representação em Árvore

$$r1 = (-b + \text{sqrt}(b^{**}2 - 4*a*c)) / (2*a)$$

6



A representação em árvore determina a ordem de avaliação da expressão aritmética?

Sequenciamento com EA

Sintaxe para expressões

7

requer escrever árvores como sequência linear de símbolos:

$$r1 = (-b + \sqrt{b^2 - 4ac}) / (2a)$$

- 1) Notação **prefixada**: operador, op. esquerda → direita

- **Polonesa de Cambridge** (utilizada em Lisp)

$$(/ (+ (-b) (\sqrt{(- (^ b 2) (* (* 4 a) c) }))) (* 2 a))$$

- **Polonesa** (Lukasiewicz)

$$/ + - b \sqrt{ - ^ b 2 * * 4 a c * 2 a }$$

- 2) Notação **pós-fixada** ou **polonesa reversa**

$$b - b 2 ^ 4 a * c * - \sqrt{ + 2 a * / }$$

- 3) Notação **infixada**

$$(-b + \sqrt{b^2 - 4ac}) / 2a$$

<expr. booleana>? <cmd1>: <cmd2>

Sequenciamento com EA

Semântica das expressões

8

- O algoritmo que o tradutor de uma LP usa para avaliar expressões aritméticas determina a semântica utilizada.
- ▣ Qualquer dos formatos de expressão pode ser utilizado na representação interna de uma expressão aritmética.
 - ⇒ avaliação:
 - prefixada
 - pós-fixada
 - infixada

Semântica das Expressões

Avaliação prefixada

9

- Aridade dos operadores conhecida $\Rightarrow$ **única varredura**. Senão os parênteses são necessários.

Seja P uma EA consistindo de operadores e operandos:

1. se $(op = \text{next}(P))$ é operador então $\text{push}(op)$;
 $n = \text{número}(\text{operandos}, op)$, $N_{op} = 0$;
2. se $op_n = \text{next}(P)$ é operando então $\text{push}(op_n)$;
 $N_{op} = N_{op} + 1$;
3. Se $N_{op} = n$, então substitua os n operandos e operador pela avaliação de $(op \ op_{n1}, \ op_{n2}, \dots, \ op_{nn})$

obs: requer batimento do número de operandos.

Notação prefixada é genérica (para qualquer número de operandos).

Semântica das Expressões

Avaliação Pós-fixada

10

- Seja P uma EA em notação pós-fixada:
 1. se $op_n = \text{next}(P)$ é um operando então $\text{push}(op_n)$;
 2. se $op = \text{next}(P)$ é um operador n -ário então seus n argumentos devem estar no topo da pilha. Substitua-os pelo resultado de aplicar o operador op a eles.
- Observações:
 - esta estratégia é usada para geração de códigos em muitos tradutores de LP.
 - se $\text{next}(P)$ é um operador então todos os seus operandos já foram avaliados e empilhados.

Semântica das Expressões

Avaliação infixada

11

- Embora a notação infixada seja comum, seu uso em computação leva a ambigüidades:
 - ▣ precedência entre os operadores deve estar definida na LP ou explicitado pelo programador:
 $2*3+4 = (2*3)+4$ ou $2*(3+4)$?
 - ▣ associatividade dos operadores também deve estar definida na LP ou explicitada pelo programador:
 $2-3-4 = (2-3)-4 = -5$ ou $2-(3-4) = 3$?
 - o mais comum é à esquerda: $a-b-c = (a-b)-c$
 - há casos em que deve ser à direita: $4^3^2 = 4^(3^2)$
 - ▣ é mais adequada para operadores binários.

O mais seguro é verificar os padrões que a LP adota!

Sequenciamento com EA

Representação em run time

12

- O tradutor trabalha em dois estágios:
 1. estabelece a árvore básica de controle,
(utiliza regras implícitas de precedência e associatividade)
 2. detalha as decisões referentes à ordem de avaliação.
(gera uma sequência [otimizada] de código).

Forma executável é necessária devido a inadequação da forma infixada do programa fonte.

Sequenciamento com EA

Representação em run time

13

Formas mais fáceis para decodificação de EA são:

1. sequência em **código de máquina**
 - a) EA traduzida para código de máquina usando os 2 estágios
 - b) máquinas convencionais usam armazenamento temporário para tratar resultados **intermediários**,
 - c) representação em código de máquina possibilita interpretação por hardware, muito mais rápida.

Forma executável é necessária devido a inadequação da forma infixada do programa fonte.

Representação em run time

Formas fáceis para decodificação de EA

14

2. Estrutura em árvore:

- ▣ nessa representação, obtida ao final do estágio 1, a EA pode ser avaliada usando um interpretador por software.
- ▣ execução (estágio 2) implica percorrer a árvore.
 - em Lisp o programa inteiro tem uma estrutura em árvore.

Representação em run time

Formas fáceis para decodificação de EA

15

3. Forma **pós-fixada e prefixada**:

- ▣ a representação em run time é obtida executando os dois estágios em um único passo.
- ▣ a EA pode ser avaliada por um algoritmo simples, visto anteriormente.
- ▣ em máquinas com organização baseada em pilha, o código é representado em forma pós-fixada.

Como as estruturas em árvores de EA são avaliadas?

Avaliação da expressão

Representada em árvore

16

- O tradutor opera em duas fases:
 - 1) **geração** da representação em **árvore**.
 - 2) determinação da **ordem** de **avaliação**
(sequência de códigos primitivos)
- Esta **segunda fase** apresenta os problemas abaixo relacionados à ordem de avaliação dos termos da EA:
 - 1) regra de avaliação uniforme,
 - 2) efeitos colaterais,
 - 3) condições de erros,
 - 4) expressões booleanas com curto circuito.

Problema 1: regra de avaliação uniforme

17

□ Utiliza somente a Regra de avaliação uniforme ou a gulosa

□ **Regra Gulosa** (passagem de parâmetros por valor):

- 1) para cada nó operador, avalie seus nós operandos,
- 2) aplique o operador aos operandos avaliados.

□ A ordem de avaliação dos operandos ou de operações independentes pode ser escolhida para otimizar o uso de temporários.

- Ex.: $(a+b)*(c-a)$: somar ou subtrair primeiro leva ao mesmo resultado.

□ A uniformidade gulosa, **nem sempre é desejável.**

- Ex.: function F(a:boolean; b, c:integer): integer
begin if a then F := b else F := c; end;

.....

Z := F(X==0, X, **Y/X**); - gera a divisão por zero

Problema 1: regra de avaliação uniforme

18

- **Regra preguiçosa** (passagem de parâmetros por referência):
 - ▣ 1) aplique o operador sobre os operandos não avaliados,
 - ▣ 2) só avalie uma expressão quando ela for requerida.
- Avaliação preguiçosa requer uma substancial simulação por software, o que pode torná-la uma sobrecarga para algumas LP como C e Fortran.
- A avaliação preguiçosa é o **padrão em LP funcionais puras**, como Haskell, Goffe e Hugs.

Regra de avaliação uniforme

19

- Avaliação gulosa e preguiçosa **são por si SÓ inadequadas**
- Uma **mistura** dessas regras parece mais ser **adequada**:
 - ▣ duas categorias de funções Lisp: as que recebem operandos já avaliados e as que recebem sem avaliação.
 - ▣ Algol: suas **primitivas são gulosas**. Operações definidas pelo usuário são gulosas ou preguiçosas (**by name**)

Problema 2: efeitos colaterais

20

- Seja a expressão $y * f(x) + y$.

A **ordem** de avaliação pode **alterar** seu **resultado**?

1) Não, se $f()$ não tem efeito colateral sobre y .

2) Sim, se **$f()$** tem efeito colateral sobre **y** .

- Suponha que $y = 2$ e $f(x)$ retorne o valor 5. Mas a **execução** de **$f()$** altera o valor de **$y = 3$** . Considere:

1) avaliar cada termo na sequência: $2 * 5 + 3 = 13$

2) avaliar y apenas uma vez: $2 * 5 + 2 = 12$

3) avaliar $f(x)$ antes de y : $3 * 5 + 3 = 18$

- Esses valores são válidos de acordo com a sintaxe da LP.

Problema 2: efeitos colaterais

21

- Há duas posições sobre efeitos colaterais:
 - 1) não permiti-los e avaliar de modo otimizado as expressões.
 - 2) permiti-los e avaliar as expressões na ordem declarada, deixando a **responsabilidade** da repercussão de efeito colateral com o **programador**.

Problema 3: condições de erros

22

- ❑ Falha de operações gerando condições de erros (**divide by zero, overflow, underflow**)
- ❑ Como tais erros podem ser decorrentes da **ordem de avaliação**, o programador precisa poder definir a ordem de avaliação usada. Isto **impede otimizar** o código.

$$X = 1.0E+20 * 5.5E+20 / 8.2E+20$$

- ❑ A solução de tais dificuldades é ad hoc e varia de LP para LP e de implementação para implementação.

Problema 4: expressões lógicas em curto-circuito

23

- Seja as operações booleanas:

1) $b1 \text{ or } b2$, se $b1 = \text{true}$, $b1 \mid b2 = \text{true}$

2) $b3 \text{ and } b4$, se $b3 = \text{false}$, $b3 \& b4 = \text{false}$

O valor de primeiro operando pode curto-circuitar toda a expressão.

- Muitos erros são gerados na **expectativa** que a LP implemente o comportamento acima,

- ▣ nem todas o fazem, em nome da “**avaliação uniforme**”

Exemplo:

1) $\text{if } ((A == 0) \mid (B/A > C)) \{ \dots \}$

2) $\text{while } ((I \leq UB) \& \& (V[I] > C)) \{ \dots \}$

UB é o upper bound (limite superior) da dimensão de um vetor

Controle de sequência entre comandos

Comandos básicos

24

- Controle de sequência determina a **ordem na qual os comandos individuais são executados.**
- Comandos básicos aplicam operações sobre os OD, alterando o estado do sistema.
- Exemplos de comandos básicos:
 - 1) Atribuição
 - liga um valor a um OD.
 - 2) Chamada de subprograma
 - liga valores aos parâmetros formais.
 - 3) Entrada e Saída
 - entrada de dados é feita para OD e a saída altera os valores da variável em um buffer.

Controle de sequência entre comandos

Controle a nível de comando

25

□ Composição

Sequência textual de comandos que são executados quando a estrutura maior que os contém for executada.

□ Alternação

Uma de duas sequência de comandos é executada quando a estrutura maior que as contém for executada.

Pode se ter alternação com **várias sequências opcionais**, das quais **apenas uma** é executada.

Controle de sequência entre comandos

Controle a nível de comando

26

□ **Iteração**

Uma sequência de comandos pode ser executada, repetidamente, zero ou mais vezes, quando a estrutura maior que a contém for executada.

□ **Controle de sequência explícito**

goto incondicional, goto condicional, break e continue.

Controle de sequência estruturada

27

- **Comando composto:** begin .. end;
É a estrutura básica para representar a **composição** de comandos.
- **Comandos condicionais**
Expressam **alternação**. As formas mais comuns de comandos condicionais são: **if** e **case**.

Controle de sequência estruturada

28

if <cond> then <comando> endif

if <cond> then <comando1> else <comando2> endif

A escolha entre muitas opções pode ser feita por aninhamento de **if**.

case tag

caso 1: comando_1;

caso 2: comando_2;

...

caso n: comando_n;

outros: comando_alternativo;

end case;

Controle de sequência estruturada

Interação

29

□ Repetição simples

perform <corpo> k times. Como em
COBOL

□ Repetição enquanto condição ou até que uma condição se estabeleça.

while <teste> do <corpo>;
repeat <corpo> until <teste>

Controle de sequência estruturada

Interação

30

- **Repetição enquanto incrementa** um contador
for i:= 1 step 2 until 30 do <corpo>;
for i:= <exp1> step <exp2> until <exp3> do
<corpo>;

Como no Algol

- **Repetição indefinida**
loop ... exit when <condition> ... end loop; (ADA)
while **true** do begin ... end;
(Pascal)

Controle de sequência estruturada

Interação

31

- C permite todos esses tipos na mesma estrutura for:

for (exp1; exp2; exp3) {corpo};

1) contador **simples** de 1 a 10

for (i=1; i<=10; i++) {corpo};

2) loop **infinito**

for (; ;) {corpo};

3) Contador com **condição de saída**

for (i=1; i<=100 &&NotEndfile; i++) {corpo};

- Em loops com **múltiplas saídas** o desvio incondicional é necessário: faz-se uso do **goto** ou **exit** (Ada) ou **break** (C).

O excelente Donald Knuth tinha razão, como sempre!